Mục lục kế hoạch ALPR

Phiên bản alpr-plan-v1 / alpr-contract-v1 / alpr-custom-base-group-v1. **Đây là kế hoạch, chưa phải dữ liệu hay mô hình đã nghiệm thu.**

Tài liệu	Người đọc	Mục đích
00 Tổng quan	Cả nhóm	MVP, nguồn tham khảo, ownership, đường găng
01 Tuần 1 A	A, B	Dữ liệu, split, crop, QA và metric
02 Tuần 2 B	B, C	Detector baseline, dev và bàn giao
03 Tuần 3 C	C, D	CRNN + CTC và pipeline tích hợp
04 Tuần 4 D	D	Demo, test, báo cáo, release
05 Gate và metric	Cả nhóm	Định nghĩa, mẫu số, ngưỡng, bằng chứng
06 Hợp đồng	Cả nhóm	Field, invariant và A → B → C → D
07 Dataset/protocol	A, D	Parser và chống rò rỉ
08 Kiến trúc/API	B, C	Shape, CTC, checkpoint, lỗi runtime
09 Tài nguyên/rủi ro	Cả nhóm	Pilot, fallback, thời gian GPU/tải
10 Tái lập/trạng thái	Cả nhóm	Lệnh hiện có và lệnh dự kiến
Decision log	Cả nhóm	Freeze, đổi protocol trước test
Nguồn tham khảo	Cả nhóm	Provenance repo mẫu và nguồn chính thức

Dữ liệu máy đọc: weekly-benchmarks.json, data-contract.json. PDF tổng hợp 38 trang gồm các tài liệu kế hoạch; biên bản kiểm tra ghi lại các lệnh đã chạy. Mẫu số đo dùng null và CHƯA ĐO; backlog và config mẫu là gợi ý. Template bàn giao, source manifest, run manifest, model manifest, release manifest, mẫu thực nghiệm. Data manifest và charset thiết kế bổ sung cho hợp đồng.

Tổng 160 giờ công: 136 thực hiện + 24 dự phòng. Không có lịch bắt đầu cụ thể. Gate toàn vẹn bắt buộc; ngưỡng chất lượng ban đầu là đề xuất có độ tin cậy thấp, phải pilot dev và freeze trước test. Không được biến một ngưỡng đã trượt thành đạt bằng cách sửa bảng kết quả hoặc đổi mẫu số.

Tổng quan dự án và trách nhiệm

Kết quả cần giao

Ảnh xe chứa một biển số → YOLOv8n → bbox → crop chữ nhật → grayscale/resize/padding → CNN → BiLSTM → logits → greedy CTC → biển số UTF-8 và trạng thái. Huấn luyện detector và recognizer riêng. Không thay CRNN bằng dịch vụ OCR. MVP demo ảnh đơn/batch. Video, tracking, camera, API production, beam search và rectification nằm ngoài đường găng.

G1 có dữ liệu đủ tin cậy; G2 có detector nạp lại được; G3 có pipeline gọi được; G4 có demo và test cuối tái lập. Gate chi tiết là chuẩn nghiệm thu. Tất cả mô hình và chất lượng hiện CHƯA ĐO.

Phân công nối tiếp

Người	Tuần	Sở hữu đầu ra	Người nhận
A	1	Nguồn, parser, split, crops, charset, metric edit distance/IoU/exact, bộ test fixtures	B; C/D dùng tiếp
B	2	YOLO baseline, adapter detect(), export predicted dev crops, scorer detection, môi trường GPU đã khóa	C
C	3	CNN + BiLSTM + CTC, recognize(), predict(), evaluator tích hợp, frozen release candidate	D
D	4	UI/CLI demo, evaluator test cuối, latency, phân tích lỗi, báo cáo, gói phát hành	Người chấm

A làm trước metric chung và input preprocessing có fixture để giảm tải tuần C; B giao adapter và predicted dev crops để C không viết lại detector. C ghép luồng suy luận ngay ngày 1–2 bằng adapter thật và CRNN chưa huấn luyện chỉ để kiểm shape; không công bố output này như kết quả. D nhận pipeline đã chạy với checkpoint thật sau G3. D không được giao nhiệm vụ viết CRNN hoặc ghép các module cốt lõi.

Mỗi tuần 34 giờ thực hiện + 6 dự phòng. Hai giờ đầu B/C/D kiểm nhận trong quỹ 34 giờ. Nếu đầu vào không hợp lệ: lập biên bản từ chối, ưu tiên dự phòng/giảm phần mở rộng, điều chỉnh lịch công khai. Không mặc định gọi người trước quay lại miễn phí; gate thiếu artifact phải giữ trạng thái CHƯA ĐẠT.

Dữ liệu và phép đo

MVP CCPD2019 ccpd_base, biển 7 ký tự. Custom split theo nhóm chuỗi biển số và cụm ảnh trùng, seed 42, mục tiêu 20.000/2.000/2.000 train/dev/test; tối thiểu 5.000/500/500. Pilot riêng 1.000 train + 200 dev lấy từ train/dev đã khóa; không tạo test pilot. Nếu thiếu nhóm để đạt quy mô tối thiểu phải báo blocker, không cho phép overlap. Các subset khó và CCPD-Green 8 ký tự chỉ mở rộng khi đủ thời gian/GPU. Chi tiết trong protocol.

Ba chế độ đánh giá độc lập: OCR trên GT bbox crop (không warp), OCR trên predicted crop có điều kiện, pipeline toàn ảnh. Không gộp oracle GT corners vào end-to-end thực tế. Dev dùng chọn confidence/NMS/preprocessing/epoch; test chỉ do D mở sau freeze.

Cách làm việc

Task, lịch ngày, framework và CLI đề xuất được thay bằng decision log nếu vẫn đáp ứng đề và hợp đồng. YOLO + CNN/BiLSTM/CTC và định nghĩa CER/E2E bắt buộc giữ. Mọi release giữ hash nguồn/split/config/charset/checkpoint, log và lệnh tái lập. Backlog hoàn tất không thay cho số đo, mẫu số hay bằng chứng. Repo mẫu chỉ cung cấp tổ chức tài liệu và cơ chế bàn giao; không sao chép dữ liệu/metric tìm kiếm.

Tuần 1 — A: dữ liệu sẵn sàng

Benchmark nghiệm thu đặt trước task

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G1.1	Quy mô khả thi	>= 1 ratio	integrity
G1.2	Toàn vẹn schema/provenance	>= 1 ratio	integrity
G1.3	Giao split bị cấm	<= 0 count	integrity
G1.4	Parser audit	>= 1 ratio	integrity
G1.5	Metric/charset/preprocessing fixtures	>= 1 ratio	integrity

Định nghĩa/mẫu số/ngưỡng phấn đấu/cơ sở/lệnh của từng ID: benchmark chuẩn. Tất cả **CHƯA ĐO**.

Mục tiêu và phạm vi

A sở hữu dataset/parser/split/crop/charset/QA và metric dùng chung. B không phải viết lại parser hoặc đoán mapping.

Đầu vào và điều kiện nhận

Đầu vào: tài liệu nguồn CCPD, quyền lấy archive, disk budget, contract v1. Hai giờ đầu kiểm nguồn và quy mô pilot; chưa nhận model từ ai.

Ngân sách và task gợi ý

34 giờ thực hiện +6 dự phòng; không tính GPU chạy nền và tải mạng thành giờ công chờ máy.

Task	Gợi ý	Giờ
A01	Nguồn, dung lượng, protocol và quyền truy cập	2
A02	Parser filename, mapping, quarantine và fixture	6
A03	Dedup, group split và anti-leak inventory	7
A04	Crop/preprocess, charset và CTC shape contract	6
A05	Metric chung, QA100 và acceptance suite	7
A06	Manifest, sample offline, báo cáo và bàn giao	6

Lịch ngày tham khảo

• Ngày 1 (8h): nguồn 2h + parser 6h; xác minh 7 field và lỗi annotation.

• Ngày 2 (8h): dedup/group split 7h + crop 1h; freeze nhóm trước tạo crop.

• Ngày 3 (8h): crop/charset 5h + metrics/QA 3h.

• Ngày 4 (8h): metrics/QA 4h + manifests/sample 4h.

• Ngày 5 (8h): bàn giao 2h + dự phòng 6h.

Cách triển khai đề xuất

Parser: bbox LT/RB, corners RB/LB/LT/RT, mapping positional province/letter/ads, sentinel O bị quarantine. Giữ raw coordinates; canonical bbox half-open theo quy tắc x2/y2 không cộng 1, audit hình ảnh trước freeze. Dedup byte + decoded RGB + pHash candidate và union-find với text plate; toàn bộ connected component chỉ một split. Split seed42 trước crop/augmentation; runtime ảnh tên opaque không chứa nhãn; test gold tách khỏi consumer. Charset + hash/version, crop grayscale aspect-resize/pad, metric Levenshtein/IoU/exact và fixtures được A viết thật. QA100 train/dev và fixture lỗi phải có annotation đối chiếu độc lập, không tự dùng parser làm gold. Tạo smoke 20 ảnh train/dev và loader/offline checker có manifest; B chạy được mà không tải raw. Phân loại train labels hợp lệ với test gold: trainer được nhận GT train, evaluator giữ GT dev/test.

Đầu ra và bằng chứng

• source/data/split/charset manifests + sample/crop JSONL và YOLO label export.

• 20 ảnh smoke train/dev + nhãn, checksum; không có dữ liệu thật ở repo khởi đầu này.

• Source/QA/quarantine/leakage reports; tests parser/metric/preprocess có lệnh tái lập.

• Báo cáo A và W1_A_to_B; hướng dẫn lấy dữ liệu và thay root paths.

Tái lập dự kiến

python -m alpr.cli verify-gate --gate G1 --config configs/frozen.yaml **CHƯA TRIỂN KHAI**. Chủ sở hữu phải thay bằng lệnh đã chạy thật trong biên bản, kèm stdout/stderr, env lock, commit, seed, hashes và per-sample CSV.

Rủi ro, fallback và chưa đạt

Archive chậm hoặc disk ít: chỉ tải subset qua nguồn có thể xác minh; nếu không có tải từng subset thì dùng kho ngoài và chuyển phần Base cần thiết. Không hứa archive hỗ trợ streaming chưa kiểm. Dưới 5k/500/500 hoặc overlap >0 => G1 chưa đạt.

Checklist bàn giao

• [ ] Hash/nguồn/version, count split tối thiểu, audit100 và quarantine log đầy đủ.

• [ ] B nhận sample và validate trong 2h; test gold không nằm runtime.

• [ ] Contract/charset/preprocess/fixtures đủ cho B và C; không còn mapping chưa rõ.

Dùng template bàn giao. Người giao chuẩn bị trong tuần mình, người nhận kiểm 2h đầu tuần kế; không suy ra nghĩa vụ hỗ trợ ngoài tuần.

Tuần 2 — B: YOLO phát hiện biển số

Benchmark nghiệm thu đặt trước task

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G2.1	Detector artifact nạp lại	>= 1 ratio	integrity
G2.2	Detection precision@0.50	>= 0.8 ratio	quality
G2.3	Detection recall@0.50	>= 0.85 ratio	quality
G2.4	Detection mAP50	>= 0.8 ratio	quality
G2.5	Detection mAP50-95	>= 0.5 ratio	quality
G2.6	Bàn giao predicted crops	>= 1 ratio	integrity

Định nghĩa/mẫu số/ngưỡng phấn đấu/cơ sở/lệnh của từng ID: benchmark chuẩn. Tất cả **CHƯA ĐO**.

Mục tiêu và phạm vi

B sở hữu baseline YOLOv8n, adapter detect(), evaluator detection, predicted dev crops và artifact detector nạp lại được.

Đầu vào và điều kiện nhận

Nhận G1: data/split/hash, parser/crop/charset, smoke20 và metric fixtures. Trong 2h kiểm manifest, count, một batch label overlay; hash/split lỗi thì từ chối đầu vào.

Ngân sách và task gợi ý

34 giờ thực hiện +6 dự phòng; không tính GPU chạy nền và tải mạng thành giờ công chờ máy.

Task	Gợi ý	Giờ
B01	Kiểm nhận A trong hai giờ đầu	2
B02	Môi trường GPU, smoke và YOLO adapter	4
B03	Pilot và chọn budget huấn luyện trên dev	6
B04	Điều phối baseline train, checkpoint và resume	5
B05	Detection evaluation, P/R/AP và failure cases	7
B06	Export predicted dev crops và test adapter	4
B07	Khóa artifact, báo cáo và bàn giao C	6

Lịch ngày tham khảo

• Ngày 1 (8h): nhận 2h + môi trường/adapter 4h + pilot 2h.

• Ngày 2 (8h): pilot 4h + điều phối train 4h; GPU wall time tách giờ công.

• Ngày 3 (8h): train/checkpoint 1h + evaluation 7h.

• Ngày 4 (8h): predicted crops/adapter 4h + đóng gói/báo cáo 4h.

• Ngày 5 (8h): bàn giao 2h + dự phòng 6h.

Cách triển khai đề xuất

Chọn YOLOv8n detection một lớp plate, Ultralytics 8.3.0, image size 640, pretrained COCO chỉ làm khởi tạo. Checkpoint nguồn phải có URL/hash và license; chưa có checkpoint nào trong skeleton. Disable augmentation có thể đảo chuỗi (horizontal/vertical flip) ở baseline; không đọc nhãn dev/test lúc train. Pilot 1k train/200 dev, 3 epochs để đo memory/throughput rồi đề xuất <=30 epochs trên subset đã chọn, early stop patience5, seed42. Không chạy sweep; chọn best theo dev mAP50-95, tie chọn epoch sớm. Budget tối đa 12 GPU giờ dự kiến; nếu pilot dự báo vượt thì giảm train subset/batch hoặc epochs và ghi log trước test. Adapter trả bbox đã undo letterbox về ảnh gốc. Test clamp bounds, score/NMS, empty case, mismatch checkpoint. Export predicted dev crops bằng cùng runtime top1 policy; giữ bbox/score/hash/coverage/parent ID; evaluator dùng GT cho chấm match, inference không được thấy GT. Không export test crops để C tune. Giữ raw detection predictions để AP dùng nhiều bbox và threshold thấp riêng với top1 pipeline.

Đầu ra và bằng chứng

• Detector checkpoint + model manifest + dependency/platform lockfile đã smoke.

• detect() adapter, YOLO data config theo root tương đối, dev predictions và P/R/AP raw evidence.

• Predicted dev crops cùng policy và ID map; không nhầm GT crop.

• Báo cáo B, pilot timing, W2_B_to_C và lệnh reload offline.

Tái lập dự kiến

python -m alpr.cli verify-gate --gate G2 --config configs/frozen.yaml **CHƯA TRIỂN KHAI**. Chủ sở hữu phải thay bằng lệnh đã chạy thật trong biên bản, kèm stdout/stderr, env lock, commit, seed, hashes và per-sample CSV.

Rủi ro, fallback và chưa đạt

VRAM ít: batch16→8→4, image640 giữ baseline trừ khi decision đổi trước test. CPU chỉ smoke; không cam kết huấn luyện đủ trên CPU. Không đạt detection quality sau dev pilot phải ghi fail/đổi lịch, không chuyển detector hỏng cho C.

Checklist bàn giao

• [ ] G2 và hashes đúng data-v1; adapter detect() callable và offline reload.

• [ ] C chạy một batch trong 2h, output coords gốc và crop đúng schema.

• [ ] Known failures/confidence/NMS/preprocessing và runtime policy được freeze hoặc có decision rõ.

Dùng template bàn giao. Người giao chuẩn bị trong tuần mình, người nhận kiểm 2h đầu tuần kế; không suy ra nghĩa vụ hỗ trợ ngoài tuần.

Tuần 3 — C: CRNN và pipeline hoàn chỉnh

Benchmark nghiệm thu đặt trước task

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G3.1	CRNN đúng kiến trúc/CTC	>= 1 ratio	integrity
G3.2	OCR GT crop CER	<= 0.2 edit/char	quality
G3.3	OCR GT crop exact	>= 0.55 ratio	quality
G3.4	OCR predicted crop CER	<= 0.25 edit/char	quality
G3.5	OCR predicted crop exact	>= 0.45 ratio	quality
G3.6	Pipeline End-to-End Accuracy	>= 0.4 ratio	quality
G3.7	Pixel-only/freeze/coverage	>= 1 ratio	integrity

Định nghĩa/mẫu số/ngưỡng phấn đấu/cơ sở/lệnh của từng ID: benchmark chuẩn. Tất cả **CHƯA ĐO**.

Mục tiêu và phạm vi

C sở hữu CNN + BiLSTM + CTC Loss, greedy CTC, recognize(), predict(), evaluator tích hợp và freeze. D phải nhận pipeline thật có thể gọi.

Đầu vào và điều kiện nhận

Nhận G2 + G1 artifacts: detector callable/checkpoint, predicted dev crops, GT train/dev crops, charset và CTC fixtures. Trong 2h verify toàn bộ hash và shape; thiếu adapter hoặc charset thì chưa nhận.

Ngân sách và task gợi ý

34 giờ thực hiện +6 dự phòng; không tính GPU chạy nền và tải mạng thành giờ công chờ máy.

Task	Gợi ý	Giờ
C01	Kiểm nhận detector, charset và data hash	2
C02	CNN, BiLSTM, CTC trainer và greedy decoder	8
C03	Ghép predict() và trạng thái lỗi từ đầu tuần	4
C04	Pilot/huấn luyện và OCR dev GT/predicted	7
C05	Hai cấu hình preprocessing và lỗi detector/OCR	5
C06	Freeze checkpoint/config, evaluator và anti-leak	4
C07	Smoke offline, báo cáo và bàn giao pipeline D	4

Lịch ngày tham khảo

• Ngày 1 (8h): nhận 2h + CRNN 4h + pipeline 2h.

• Ngày 2 (8h): CRNN 4h + pipeline 2h + pilot train 2h.

• Ngày 3 (8h): train/dev 5h + ablation 3h.

• Ngày 4 (8h): ablation 2h + freeze/evaluator 4h + handoff 2h.

• Ngày 5 (8h): handoff 2h + dự phòng 6h.

Cách triển khai đề xuất

Dùng thiết kế CRNN cụ thể tại 08: grayscale 32x160, CNN width stride4, T40, BiLSTM2 tầng hidden128, projection C=68 (67 token thật + blank0), log_softmax, CTCLoss. Không pretrained OCR thay đề bài. Train GT bbox crops train, dev GT crops chọn best CER; teacher labels train hợp lệ. Greedy decode argmax→collapse repeats→remove blank; không sửa chuỗi theo GT/filename. Test repeated labels, target lengths và T>=L+adjacent-repeats; zero_infinity không che invalid CTC samples. Pilot 1k train/200 dev, 2 epochs, sau đó <=40 epochs, patience5; tối đa 16 GPU giờ dự kiến. Tối đa hai cấu hình preprocess: baseline grayscale và CLAHE (clip2,tile8x8), chọn trên dev rồi khóa; không train thêm biến thể nếu budget đã vượt, báo thiếu ablation trung thực. Đánh giá GT crop, predicted conditional và original-image pipeline, cùng denominators. Gắn failure status; test đổi tên cùng pixel và opaque ID không làm thay prediction ngoài trường ID. Freeze checkpoint/config/charset/split và cả ablation artifacts trước khi D mở test. Pipeline không có parser filename hoặc nhận GT; nhận pixel qua RuntimeImage. Gói bàn giao phải có predict() thật, sample20, evaluator, dependency lock và smoke log clean checkout.

Đầu ra và bằng chứng

• Recognizer state_dict/checkpoint, optimizer resume riêng nếu cần, decoder version, model manifest.

• recognize() + predict() callable, tests interface/CTC/anti-leak, evaluator toàn split.

• Dev raw results, ablation manifests/configs, frozen run/release candidate manifest.

• W3_C_to_D, báo cáo C, demo smoke sample không nhãn trong tên runtime.

Tái lập dự kiến

python -m alpr.cli verify-gate --gate G3 --config configs/frozen.yaml **CHƯA TRIỂN KHAI**. Chủ sở hữu phải thay bằng lệnh đã chạy thật trong biên bản, kèm stdout/stderr, env lock, commit, seed, hashes và per-sample CSV.

Rủi ro, fallback và chưa đạt

CTC không hội tụ: kiểm lengths/blank/charset/input, overfit 32 train samples để debug rồi bỏ run debug khỏi kết quả. Giảm subset/epochs hoặc bỏ extension; không thay CRNN. Chưa có checkpoint/predict() callable => G3 chưa đạt và D không bắt đầu ghép core.

Checklist bàn giao

• [ ] D gọi predict() bằng pixel trong 2h với đúng checkpoint/hash.

• [ ] Metric full split/miss và CTC repeated-label fixtures pass.

• [ ] Freeze trước test, threshold decision có dev evidence, GT không lọt inference.

Dùng template bàn giao. Người giao chuẩn bị trong tuần mình, người nhận kiểm 2h đầu tuần kế; không suy ra nghĩa vụ hỗ trợ ngoài tuần.

Tuần 4 — D: demo, test và phát hành

Benchmark nghiệm thu đặt trước task

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G4.1	Demo ảnh đơn/batch và lỗi	>= 1 ratio	integrity
G4.2	Test End-to-End Accuracy	>= 0.4 ratio	quality
G4.3	Test pipeline CER	<= 0.35 edit/char	quality
G4.4	Joint detection-recognition@0.50	bắt buộc báo số đo	reporting
G4.5	Báo đầy đủ detection/OCR/coverage/strata	>= 1 ratio	integrity
G4.6	Latency và tài nguyên	bắt buộc báo số đo	reporting
G4.7	Ablation/error analysis/release	>= 1 ratio	integrity

Định nghĩa/mẫu số/ngưỡng phấn đấu/cơ sở/lệnh của từng ID: benchmark chuẩn. Tất cả **CHƯA ĐO**.

Mục tiêu và phạm vi

D làm demo ảnh đơn/batch, final test, latency, phân tích lỗi và gói nộp; không viết mới detector/CRNN/pipeline.

Đầu vào và điều kiện nhận

Nhận G3: predict() callable, checkpoint thật, manifest freeze, sample20, evaluator và ablation artifacts. Trong 2h reload offline và kiểm hash; test labels chỉ mở sau bước này.

Ngân sách và task gợi ý

34 giờ thực hiện +6 dự phòng; không tính GPU chạy nền và tải mạng thành giờ công chờ máy.

Task	Gợi ý	Giờ
D01	Kiểm nhận pipeline đã ghép và frozen manifest	2
D02	Demo ảnh đơn và batch, xử lý lỗi	6
D03	Test cuối từ frozen artifacts và per-image metrics	6
D04	Latency, resource và clean-machine reproduction	5
D05	Strata, ablation và audit lỗi	5
D06	Báo cáo, slide và kịch bản demo	6
D07	Release manifest, README và gói nộp	4

Lịch ngày tham khảo

• Ngày 1 (8h): nhận 2h + demo 6h.

• Ngày 2 (8h): test cuối 6h + latency/reproduce 2h.

• Ngày 3 (8h): latency/reproduce 3h + strata/ablation/error audit 5h.

• Ngày 4 (8h): báo cáo/slide/demo 6h + release 2h.

• Ngày 5 (8h): release 2h + dự phòng 6h.

Cách triển khai đề xuất

Demo nhận upload pixel, tên file chỉ để hiển thị tùy chọn sau inference; không dùng CCPD parser. Ghi bbox/text/score/status riêng, ảnh lỗi hiển thị lỗi rõ; batch giữ 1 result cho mọi input. Sáu fixture demo: ảnh đơn hợp lệ; batch có ảnh lỗi; no_detection; decode error; ocr_empty; manifest mismatch. D giữ test gold ngoài runtime, chấm toàn frozen test đúng ID, không lọc ảnh fail và không tune bằng test. Báo detection P/R/AP, OCR GT/predicted/coverage, E2E/CER/joint, strata và failure counts. Ablation cùng tập/config đã freeze; GT corners nếu có chỉ là oracle bảng riêng. Đo latency đúng warmup/batch/synchronization, hardware, I/O/model/pipeline riêng. Audit <=50 lỗi có taxonomy và hình minh họa; không công bố dữ liệu cá nhân ngoài phạm vi đồ án. Nếu phát hiện bug evaluator/pipeline sau test: log lần exposure, sửa correctness, rerun mọi run bị ảnh hưởng; không dùng lần sửa để chọn augmentation/decoder mới. Kết quả test trở thành exposed, nêu rõ trong báo cáo. Báo cáo có phương pháp, tài nguyên, split custom, hạn chế domain/identity và số đo thật; slide và video backup chỉ khi có demo thật.

Đầu ra và bằng chứng

• Demo offline, frozen checkpoints/manifests và asset runtime opaque đã kiểm hash.

• Per-image predictions/metrics, test exposure log, strata/error audit, latency CSV.

• Báo cáo/slide/README có trạng thái và hướng tái lập, release manifest và gói nộp.

• Biên bản G4 với pass/fail thật; không giấu ngưỡng trượt hoặc thiếu ablation.

Tái lập dự kiến

python -m alpr.cli verify-gate --gate G4 --config configs/frozen.yaml **CHƯA TRIỂN KHAI**. Chủ sở hữu phải thay bằng lệnh đã chạy thật trong biên bản, kèm stdout/stderr, env lock, commit, seed, hashes và per-sample CSV.

Rủi ro, fallback và chưa đạt

Thiếu checkpoint hoặc predict() fail smoke: từ chối G3, lập blocker/lịch mới. Demo tối giản CLI/image report thay UI nhiều tính năng. Không mở rộng video/API để bù thiếu metric. Quality trượt vẫn công bố số thật và gate fail.

Checklist bàn giao

• [ ] Test freeze/hash/denominator đúng, không tune sau test.

• [ ] 6/6 demo fixture và reload offline, không tự tải model khi chạy.

• [ ] Manifest release không vòng hash; không commit dataset/checkpoint lớn; kết quả chưa đo giữ null.

Dùng template bàn giao. Người giao chuẩn bị trong tuần mình, người nhận kiểm 2h đầu tuần kế; không suy ra nghĩa vụ hỗ trợ ngoài tuần.

Gate G1–G4 và định nghĩa phép đo

Tất cả số đo hiện CHƯA ĐO / null. Gate chỉ đạt khi có artifact, denominator, log và lệnh chạy lại. integrity là bắt buộc; quality là ngưỡng đề xuất cần pilot dev, sau freeze trở thành cam kết; reporting yêu cầu đo thật và công bố đầy đủ, chưa áp đặt ngưỡng tốc độ khi phần cứng chưa biết. Gate có ngưỡng chất lượng chưa đạt phải báo CHƯA ĐẠT dù backlog đã xong.

Nguồn ngưỡng là ngân sách 4 x40 giờ, 5k–20k ảnh train và một GPU 8 GB giả định; không sao chép kết quả bài báo. Mọi ngưỡng quality có độ chắc chắn thấp cho tới pilot. B đề xuất detection sau pilot tuần 2; C đề xuất OCR/E2E sau pilot tuần 3, D kiểm manifest freeze trước test. Chỉ chấp nhận thay đổi khi có decision log, dev evidence, lý do tài nguyên và chữ ký người nhận trước mở test. Không hiệu chỉnh trên test. Integrity không được nới.

Protocol chung

CER micro = Σ Levenshtein(pred,gt) / Σ số codepoint GT sau NFC cho cả hai. Không strip dấu, không xóa ký tự lạ, không sửa theo format biển hoặc lexicon GT. CER có thể >1. Exact plate accuracy đòi chuỗi NFC bằng hoàn toàn. Mẫu số rỗng => null + lỗi đánh giá, không thành 0 hoặc 1. Nếu prediction thiếu/duplicate ID: evaluator từ chối báo thành công, lưu lỗi inventory; runner tạo một result cho mỗi input, ghi error khi thất bại. Để tính toàn split, missing prediction được tính empty/fail và báo coverage lỗi.

Detection precision/recall dùng tất cả bbox sau NMS IoU=0.50 và confidence>=0.25 mặc định (chỉ chỉnh bằng dev). Matching theo score giảm dần, mỗi GT tối đa một match, IoU>=0.50. FP=prediction không match; FN=GT không match. Nếu TP+FP=0, precision=0 kèm no_predictions=true; recall=0 khi có GT. Ngt=0 => null. AP dùng score>=0.001, cùng NMS, tối đa 100 detections/ảnh, PR envelope 101 recall points, AP50 và mean AP ở 10 IoU 0.50:0.05:0.95, một lớp plate. Ghi evaluator/version chính xác. Các diagnostic P/R dùng nhiều bbox, còn pipeline chỉ chọn một bbox; đây là hai protocol khác nhau được ghi rõ.

Pipeline chọn top1 confidence cao nhất sau NMS và threshold runtime; hòa score thì sort (x1,y1,x2,y2) tăng dần. Không dùng IoU với GT, OCR text hoặc filename để chọn bbox. Không có top1 => no_detection. OCR GT crop dùng bbox GT chữ nhật, không warp corners. Predicted crop diagnostic chỉ dùng top1 có IoU>=0.50 và crop hợp lệ; evaluator mới dùng GT để xác định tập điều kiện. Coverage=Nmatched/Nsplit; báo thêm Nselected, Ncrop_valid, Nunmatched. Nếu không matched, conditional metric=null và gate quality fail. GT CER/exact giữ OCR lỗi/rỗng trong mẫu số; predicted CER/exact cũng giữ lỗi OCR của mọi crop trong tập matched.

End-to-End Accuracy = số ảnh có chuỗi top1 nhận đúng hoàn toàn / mọi ảnh trong split. Metric chính không đòi IoU; nếu top1 ở vị trí sai mà text tình cờ đúng vẫn đúng theo metric text này. Joint@0.50 đòi thêm IoU>=0.50, tên riêng. Miss/crop_error/ocr_empty/runtime_error luôn sai E2E. Pipeline CER dùng pred rỗng cho các lỗi đó. Không bỏ ảnh khó hoặc ảnh chạy lỗi khỏi báo cáo. Ảnh hỏng nguồn phải quarantine trước freeze split, có số lượng/lý do; ảnh lỗi phát sinh sau freeze không tự xóa khỏi denominator.

Strata: toàn Base và các bin blur/brightness/tilt/plate-area được định nghĩa bằng train trước freeze; report N, CER, E2E của từng bin, micro toàn split; nhóm rỗng có N=0 và metric=null. Nếu chạy subset DB/Blur/FN/Rotate/Tilt/Challenge/Green, báo riêng tên protocol và N từng subset, không gộp vào headline Base hoặc giả sử đã đo. Báo Wilson95% cho E2E, bootstrap seed42 1000 lần cho CER nếu còn thời gian.

Latency: đo model detect và OCR riêng trên tensor resident (đồng bộ CUDA trước/sau), pipeline từ pixel CPU qua preprocess/transfer/detect/crop/OCR/decode/serialization; báo thêm file I/O riêng. Không tính load checkpoint vào latency steady-state, báo cold start riêng. 20 warm-up +200 lượt đo, batch=1, GPU synchronize, no output cache, một checkpoint/config cố định. p95 là nearest-rank sort[ceil(.95*N)-1], p50 tương tự; báo throughput batch riêng nếu đo. Record GPU/CPU/driver/CUDA, RAM, peak allocated/reserved VRAM, torch/package versions, precision và model hash.

Bảng gate

G1

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G1.1	Quy mô khả thi	>= 1 ratio	integrity
G1.2	Toàn vẹn schema/provenance	>= 1 ratio	integrity
G1.3	Giao split bị cấm	<= 0 count	integrity
G1.4	Parser audit	>= 1 ratio	integrity
G1.5	Metric/charset/preprocessing fixtures	>= 1 ratio	integrity

G2

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G2.1	Detector artifact nạp lại	>= 1 ratio	integrity
G2.2	Detection precision@0.50	>= 0.8 ratio	quality
G2.3	Detection recall@0.50	>= 0.85 ratio	quality
G2.4	Detection mAP50	>= 0.8 ratio	quality
G2.5	Detection mAP50-95	>= 0.5 ratio	quality
G2.6	Bàn giao predicted crops	>= 1 ratio	integrity

G3

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G3.1	CRNN đúng kiến trúc/CTC	>= 1 ratio	integrity
G3.2	OCR GT crop CER	<= 0.2 edit/char	quality
G3.3	OCR GT crop exact	>= 0.55 ratio	quality
G3.4	OCR predicted crop CER	<= 0.25 edit/char	quality
G3.5	OCR predicted crop exact	>= 0.45 ratio	quality
G3.6	Pipeline End-to-End Accuracy	>= 0.4 ratio	quality
G3.7	Pixel-only/freeze/coverage	>= 1 ratio	integrity

G4

ID	Benchmark	Chiều / ngưỡng tối thiểu	Loại
G4.1	Demo ảnh đơn/batch và lỗi	>= 1 ratio	integrity
G4.2	Test End-to-End Accuracy	>= 0.4 ratio	quality
G4.3	Test pipeline CER	<= 0.35 edit/char	quality
G4.4	Joint detection-recognition@0.50	bắt buộc báo số đo	reporting
G4.5	Báo đầy đủ detection/OCR/coverage/strata	>= 1 ratio	integrity
G4.6	Latency và tài nguyên	bắt buộc báo số đo	reporting
G4.7	Ablation/error analysis/release	>= 1 ratio	integrity

Từng benchmark: công thức, mẫu số, bằng chứng

G1.1 — Quy mô khả thi

• Chủ sở hữu: **A**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **train/dev/test**.

• Công thức: min(Ntrain/5000,Ndev/500,Ntest/500); mẫu số/số mẫu: Tất cả ảnh hợp lệ sau dedup và group split.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: 4; loại integrity.

• Cơ sở: Tối thiểu 5000/500/500; mục tiêu 20000/2000/2000. Đây là ngân sách subset, không là số lượng dataset chính thức. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g1/G1.1-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G1 --benchmark G1.1 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G1.2 — Toàn vẹn schema/provenance

• Chủ sở hữu: **A**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **all**.

• Công thức: N_valid / N_artifact_records; mẫu số/số mẫu: Mọi record và artifact bàn giao; ảnh loại có log riêng.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g1/G1.2-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G1 --benchmark G1.2 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G1.3 — Giao split bị cấm

• Chủ sở hữu: **A**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **all**.

• Công thức: N_cross_split_exact + N_cross_split_cluster + N_cross_split_plate; mẫu số/số mẫu: So sánh mọi cặp split trên toàn inventory.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: <=; đơn vị count.

• Ngưỡng tối thiểu: 0; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g1/G1.3-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G1 --benchmark G1.3 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G1.4 — Parser audit

• Chủ sở hữu: **A**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **train/dev + synthetic**.

• Công thức: N_correct / N_audited; mẫu số/số mẫu: 100 ảnh train/dev: 80 random seed42 +20 khó; >=12 fixture lỗi.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: BBox/text đối chiếu nguồn và ảnh; test không dùng audit. Trường hợp không rõ phải quarantine, không xác nhận đúng bằng parser tự so với chính nó. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g1/G1.4-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G1 --benchmark G1.4 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G1.5 — Metric/charset/preprocessing fixtures

• Chủ sở hữu: **A**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **synthetic + train/dev**.

• Công thức: N_pass / N_cases; mẫu số/số mẫu: Ít nhất 20 ca parser/CTC/metrics/shape; thống kê coverage từng nhóm.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g1/G1.5-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G1 --benchmark G1.5 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.1 — Detector artifact nạp lại

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev smoke**.

• Công thức: N_pass / N_checks; mẫu số/số mẫu: >=20 ảnh dev + manifest/hash/config/offline reload checks.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.1-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.1 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.2 — Detection precision@0.50

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev**.

• Công thức: TP/(TP+FP); mẫu số/số mẫu: Tất cả prediction sau NMS, conf cố định; TP matching 1:1 IoU>=0.50.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.8; phấn đấu: 0.9; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.2-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.2 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.3 — Detection recall@0.50

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev**.

• Công thức: TP/(TP+FN); mẫu số/số mẫu: Tất cả GT trên dev; Ndev>=500; miss vẫn là FN.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.85; phấn đấu: 0.93; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.3-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.3 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.4 — Detection mAP50

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev**.

• Công thức: AP at IoU 0.50; mẫu số/số mẫu: Một lớp plate; mọi prediction score>=0.001; Ndev>=500.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.8; phấn đấu: 0.9; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.4-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.4 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.5 — Detection mAP50-95

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev**.

• Công thức: mean(AP_t), t=0.50:0.05:0.95; mẫu số/số mẫu: Một lớp, tất cả dev; 10 IoU thresholds; 101-point interpolated PR.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.5; phấn đấu: 0.65; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.5-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.5 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G2.6 — Bàn giao predicted crops

• Chủ sở hữu: **B**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev**.

• Công thức: N_valid_linked / N_eligible; mẫu số/số mẫu: Mọi top1 detection đủ conf và crop hợp lệ; báo Neligible/Ndev.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g2/G2.6-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G2 --benchmark G2.6 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.1 — CRNN đúng kiến trúc/CTC

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **synthetic/train/dev**.

• Công thức: N_pass / N_cases; mẫu số/số mẫu: >=8 CTC/shape/checkpoint tests; CNN+BiLSTM+CTCLoss được inspect.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.1-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.1 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.2 — OCR GT crop CER

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev GT bbox crops**.

• Công thức: sum(edit_distance)/sum(len(gt)); mẫu số/số mẫu: Tất cả Ndev>=500, >=3500 ký tự GT; không loại OCR rỗng/lỗi.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: <=; đơn vị edit/char.

• Ngưỡng tối thiểu: 0.2; phấn đấu: 0.1; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.2-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.2 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.3 — OCR GT crop exact

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev GT bbox crops**.

• Công thức: N_exact / Ndev; mẫu số/số mẫu: Tất cả Ndev; NFC nguyên chuỗi; empty/error sai.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.55; phấn đấu: 0.75; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.3-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.3 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.4 — OCR predicted crop CER

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev conditional predicted crops**.

• Công thức: sum(edit_distance)/sum(len(gt_matched)); mẫu số/số mẫu: Top1 IoU>=0.50 và crop hợp lệ; báo Nmatched,Nchars,coverage; Nmatched=0 => null/fail.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: <=; đơn vị edit/char.

• Ngưỡng tối thiểu: 0.25; phấn đấu: 0.15; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.4-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.4 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.5 — OCR predicted crop exact

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev conditional predicted crops**.

• Công thức: N_exact_matched / Nmatched; mẫu số/số mẫu: Cùng tập G3.4; coverage=Nmatched/Ndev; không bỏ nhận dạng rỗng/lỗi.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.45; phấn đấu: 0.65; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.5-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.5 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.6 — Pipeline End-to-End Accuracy

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev original images**.

• Công thức: N_images_exact / Ndev; mẫu số/số mẫu: Toàn Ndev>=500; miss/crop error/runtime error = failure; top1 cố định.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.4; phấn đấu: 0.65; loại quality.

• Cơ sở: Mốc khởi đầu xấp xỉ 0.85 recall x 0.45 OCR conditional; không phải kết quả độc lập hay bảo đảm. Hiệu chỉnh bằng pilot dev. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.6-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.6 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G3.7 — Pixel-only/freeze/coverage

• Chủ sở hữu: **C**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev + synthetic**.

• Công thức: N_pass / N_checks; mẫu số/số mẫu: 100 ảnh đổi tên cùng pixel; full ID coverage; runtime không parser/GT; config+checkpoint freeze.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g3/G3.7-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G3 --benchmark G3.7 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.1 — Demo ảnh đơn/batch và lỗi

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **demo riêng**.

• Công thức: N_pass / 6; mẫu số/số mẫu: 6 ca: ảnh đơn, batch, không detect, ảnh lỗi, OCR empty, artifact mismatch.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.1-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.1 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.2 — Test End-to-End Accuracy

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **test original images**.

• Công thức: N_images_exact / Ntest; mẫu số/số mẫu: Toàn Ntest>=500; ít nhất 3500 ký tự GT; miss và mọi lỗi vẫn nằm mẫu số.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 0.4; phấn đấu: 0.65; loại quality.

• Cơ sở: Kế thừa mục tiêu dev G3.6 đã freeze; không giảm ngưỡng sau xem test. Khoảng Wilson 95% được báo kèm. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.2-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.2 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.3 — Test pipeline CER

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **test original images**.

• Công thức: sum(edit_distance(pred_or_empty,gt))/sum(len(gt)); mẫu số/số mẫu: Mọi ảnh test; miss/error pred=""; không lọc ký tự sai hoặc unknown.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: <=; đơn vị edit/char.

• Ngưỡng tối thiểu: 0.35; phấn đấu: 0.2; loại quality.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.3-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.3 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.4 — Joint detection-recognition@0.50

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **test original images**.

• Công thức: N_top1_IoU_ge_0.50_and_exact / Ntest; mẫu số/số mẫu: Toàn Ntest; metric bổ sung, khác End-to-End Accuracy.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: report; đơn vị ratio.

• Ngưỡng tối thiểu: null; phấn đấu: null; loại reporting.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.4-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.4 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.5 — Báo đầy đủ detection/OCR/coverage/strata

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **test**.

• Công thức: N_present / N_required; mẫu số/số mẫu: 11 metric bắt buộc + bảng theo từng strata quan sát được; nhóm rỗng ghi N=0, metric=null.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: 11 metric: precision, recall, mAP50, mAP50-95, GT CER/exact, predicted CER/exact/coverage, E2E, pipeline CER; joint báo thêm G4.4. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.5-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.5 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.6 — Latency và tài nguyên

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev fixed latency set**.

• Công thức: p50,p95 of elapsed_ms; mẫu số/số mẫu: 20 warm-up +200 measured forwards, batch1; nạp model một lần; 200 ID cố định từ dev; công bố CPU/GPU/VRAM/RAM.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: report; đơn vị ms.

• Ngưỡng tối thiểu: null; phấn đấu: null; loại reporting.

• Cơ sở: Mốc pilot thực dụng trên custom Base; chưa có số đo nội bộ, độ chắc chắn thấp. Cần freeze bằng dev trước test. Độ chắc chắn low_until_dev_pilot.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.6-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.6 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

G4.7 — Ablation/error analysis/release

• Chủ sở hữu: **D**; dataset: CCPD2019 ccpd_base / alpr-custom-base-group-v1; split: **dev selection + final test report**.

• Công thức: N_pass / N_checks; mẫu số/số mẫu: GT vs predicted crop; baseline vs 1 preprocessing đã chọn trên dev; >=50 lỗi hoặc toàn bộ nếu <50; release reload offline.

• Tổng hợp: micro/full split unless explicitly conditional; chiều: >=; đơn vị ratio.

• Ngưỡng tối thiểu: 1; phấn đấu: null; loại integrity.

• Cơ sở: Điều kiện tính đúng và tái lập, không được nới để che lỗi. Độ chắc chắn high.

• Tài nguyên: Python 3.11, 1 GPU CUDA 8 GB đề xuất; chưa xác minh. Subset tối thiểu nếu thiếu tài nguyên.

• Bằng chứng dự kiến: reports/runs/g4/G4.7-evidence.json + per-image/per-crop CSV, frozen manifest và stdout/stderr.

• Lệnh dự kiến **CHƯA TRIỂN KHAI**: python -m alpr.cli verify-gate --gate G4 --benchmark G4.7 --config configs/frozen.yaml.

• Kết quả hiện tại: **CHƯA ĐO**, observed=null, observed_n=null.

Nghiệm thu và ablation

Mọi bảng phải có ID run, dataset/split/config/charset/checkpoint hashes, N thực tế, numerator và denominator, failure counts, raw predictions, runtime/error log. Không dùng 0 thay số đo chưa chạy. Mẫu CSV dùng token null; JSON dùng null thật. Khi có số đo, tạo file run riêng, giữ template nguyên trạng. Nếu ngưỡng tối thiểu trượt: báo số thật, đánh dấu gate fail, đề xuất scope/schedule có decision; không chọn test đẹp.

Ablation bắt buộc vừa sức: (1) cùng recognizer trên GT crop và predicted crop, báo coverage và pipeline toàn ảnh; (2) baseline grayscale so với duy nhất một cải tiến contrast/CLAHE, chọn bằng dev với tối đa 2 cấu hình; (3) taxonomy detector miss/localization/crop/recognition, đếm theo toàn split và audit tối đa 50 lỗi. C chuẩn bị checkpoint/config cả hai biến thể đã freeze để D chạy báo cáo, không chuyển tuning sang D. Nếu không kịp biến thể cải tiến, ghi ablation chưa hoàn thành và gate tương ứng chưa đạt; không giả số đo.

Hợp đồng dữ liệu và bàn giao A → B → C → D

Phiên bản **alpr-contract-v1**. data-contract.json là JSON Schema Draft2020-12, có $defs cho từng record. Đây là đặc tả phải triển khai, không phải xác nhận dataset đã tồn tại. Tất cả field trong mỗi record đều required; null chỉ khi schema cho phép; không dùng "None" hoặc 0 thay null. additionalProperties=false: đổi field phải đổi contract version hoặc decision migration. JSON không NaN/Infinity, UTF-8 LF; text NFC; đơn vị tọa độ pixel trừ label YOLO normalized.

Ownership và nhận bàn giao

Điểm nối	Đầu ra phải có	Kiểm nhận trong 2h	Trách nhiệm khi thiếu
A → B	Source/data/split/charset manifests; annotations; YOLO labels; GT train/dev crops; metric fixtures; smoke20	Hash, PK/FK, no-overlap, bbox overlay và một batch	G1 chưa đạt; lên lịch sửa chính thức, không mặc định A quay lại miễn phí
B → C	detect() callable; checkpoint/hash; env lock; dev detections + predicted crops; G1 bundle giữ nguyên	Reload offline, undo letterbox, crop IDs và coverage	G2 chưa đạt nếu thiếu artifact; C không tự viết lại detector
C → D	recognize()/predict() callable; CRNN checkpoint; frozen configs/evaluator; ablation artifacts; sample20	Pixel-only inference, hashes, failure states và lệnh full-split	G3 chưa đạt; D không bắt đầu ghép core
D → người chấm	Demo, raw test predictions, số đo, báo cáo, release manifest	Checkout sạch, offline load, release inventory	G4 chỉ PASS khi đủ bằng chứng và ngưỡng đã freeze

Người giao viết biên bản trước hết tuần mình; người nhận xác nhận trong quỹ tuần kế. Nếu phát hiện đầu vào hỏng sau khi người giao kết thúc tuần: dùng reserve/decision đổi lịch hoặc giảm extension; không đặt nghĩa vụ hỗ trợ ẩn. Không chuyển trạng thái gate thiếu dữ liệu thành đạt.

Record và field chuẩn

Bảng dưới sinh từ cùng định nghĩa schema để giữ tên/kiểu/null đồng nhất. Annotation, Sample, OCRCrop.plate_text_nfc và source locator là dữ liệu training/evaluator; không truyền cả record này vào RuntimeImage. Trainer được nhận GT train hợp lệ, không cấm training labels. DetectorLabel xuất một dòng 0 cx cy w h cho mỗi GT plate, chuẩn hóa theo kích thước ảnh gốc, cx=(x1+x2)/(2W),cy=(y1+y2)/(2H),w=(x2-x1)/W,h=(y2-y1)/H, width/height>0.

Artifact

Đường dẫn tương đối, hash byte chính xác; không tự hash hoặc tạo vòng tham chiếu.

Field	Kiểu / null
path	string
sha256	string
size_bytes	integer
role	enum runtime,train_label,dev_gold,test_gold,evidence,checkpoint,manifest

Annotation

Chỉ trainer/evaluator sử dụng. Gốc tọa độ trái trên; bbox half-open; corners trong ảnh gốc.

Field	Kiểu / null
bbox_xyxy	array[number]
corners_rb_lb_lt_rt	array[array[number]]
raw_bbox_xyxy	array[integer]
plate_indices	array[integer]

Field	Kiểu / null
plate_text_nfc	string
source_locator	string

Sample

Record chuẩn cho training/evaluator. RuntimeImage không nhận annotation hoặc source locator.

Field	Kiểu / null
contract_version	const alpr-contract-v1
sample_id	string
image_id	string
runtime_image_path	string
image_sha256	string
pixel_sha256	string
width_px	integer
height_px	integer
subset	string
split	enum train,dev,test
dedup_group_id	string
annotation	Annotation

SplitManifest

Hash danh sách ID bằng JSON chuẩn hóa và sort ổn định; chốt trước tạo crop.

Field	Kiểu / null
contract_version	const alpr-contract-v1
protocol_version	const alpr-custom-base-group-v1
data_version	string
seed	const 42
sampling_version	string
dedup_version	string
source_manifest_sha256	string
members	object
counts	object
membership_sha256	string
quarantine_count	integer

DetectorLabel

YOLO dùng tâm/kích thước chuẩn hóa theo ảnh gốc; nhãn số chỉ ở training/evaluation.

Field	Kiểu / null
sample_id	string
image_id	string
split	enum train,dev,test
class_id	const 0
bbox_cxcywh_norm	array[number]
image_sha256	string

OCRCrop

Crop lưu cho train/evaluation; runtime crop bỏ nhãn. Crop GT có detector hash=null. Kế thừa split ảnh cha.

Field	Kiểu / null
crop_id	string
sample_id	string
image_id	string
split	enum train,dev,test
crop_kind	enum gt_bbox,pred_bbox,oracle_gt_corners,pred_corners
path	string
sha256	string
bbox_xyxy	array[number]
parent_image_sha256	string
preprocess_version	string
detector_checkpoint_sha256	string | null
plate_text_nfc	string | null

Charset

tokens[0]=<CTC_BLANK>; không có sentinel O; hash tokens theo JSON array UTF-8 compact +LF.

Field	Kiểu / null
contract_version	const alpr-contract-v1
charset_version	string
normalization	const NFC
blank_index	const 0
tokens	array[string]
tokens_sha256	string
allowed_lengths	array[enum 7,8]

DetectionPrediction

Tọa độ pixel ảnh gốc sau undo letterbox; score detector chưa hiệu chuẩn xác suất.

Field	Kiểu / null
sample_id	string
detection_id	string
class_id	const 0
bbox_xyxy	array[number]
score	number
detector_checkpoint_sha256	string

OCRPrediction

Confidence chỉ chẩn đoán, chưa hiệu chuẩn và không thay exact/CER. Output rỗng có confidence=null.

Field	Kiểu / null
sample_id	string
crop_id	string
text_nfc	string
confidence	number | null
confidence_kind	const mean_greedy_nonblank_maxprob
decoder_version	string
recognizer_checkpoint_sha256	string
charset_sha256	string
status	enum ok,ocr_empty,ocr_error

PipelineResult

Một record cho mỗi ảnh; mọi failure có text rỗng. Không có field GT. low_confidence nghĩa mọi bbox dưới ngưỡng.

Field	Kiểu / null
contract_version	const alpr-contract-v1
sample_id	string
status	enum ok,low_confidence,no_detection,crop_error,ocr_empty,ocr_error,invalid_input,runtime_error
selected_detection	DetectionPrediction | null
ocr	OCRPrediction | null
text_nfc	string
error_code	string | null
latency_ms	number | null
run_id	string

SourceManifest

Template cho phép null; READY đòi revision/nguồn archive và inventory thật.

Field	Kiểu / null
contract_version	const alpr-contract-v1
manifest_version	const alpr-manifest-v1
status	enum TEMPLATE_NOT_A_RELEASE,READY
created_at	string | null
dataset_version	const CCPD2019
upstream_url	string
upstream_commit	string | null
archive_url	string | null
archive_sha256	string | null
license_reference	string
files	array[Artifact]

DataManifest

Inventory dữ liệu sau QA; không tự hash. READY đòi counts/inventory thật.

Field	Kiểu / null
contract_version	const alpr-contract-v1
manifest_version	const alpr-manifest-v1
status	enum TEMPLATE_NOT_A_RELEASE,READY
created_at	string | null
data_version	string
source_manifest_sha256	string | null
split_manifest_sha256	string | null
charset_sha256	string | null
parser_version	string
inventory	array[Artifact]
counts	object

ModelManifest

Recognizer READY phải khóa charset/decoder và input shape qua config.

Field	Kiểu / null
contract_version	const alpr-contract-v1
manifest_version	const alpr-manifest-v1
status	enum TEMPLATE_NOT_A_RELEASE,READY
created_at	string | null
model_kind	enum detector,recognizer

Field	Kiểu / null
architecture	string
framework_version	string | null
checkpoint_sha256	string | null
data_manifest_sha256	string | null
split_manifest_sha256	string | null
config_sha256	string | null
charset_sha256	string | null
preprocess_version	string
decoder_version	string | null
artifacts	array[Artifact]

RunManifest

READY đòi ID, hash, lệnh, môi trường và bằng chứng; metric chưa đo phải giữ null và ghi rõ.

Field	Kiểu / null
contract_version	const alpr-contract-v1
manifest_version	const alpr-manifest-v1
status	enum TEMPLATE_NOT_A_RELEASE,READY
created_at	string | null
stage	enum data,detector,recognizer,pipeline
run_id	string | null
git_commit	string | null
seed	const 42
mode	enum train,dev,test,latency
source_manifest_sha256	string | null
data_manifest_sha256	string | null
split_manifest_sha256	string | null
config_sha256	string | null
charset_sha256	string | null
detector_checkpoint_sha256	string | null
recognizer_checkpoint_sha256	string | null
hardware	object
environment_lock_sha256	string | null
commands	array[string]
test_exposure	array[string]
metrics	object

Field	Kiểu / null
artifacts	array[Artifact]

ReleaseManifest

Tách runtime allowlist khỏi nhãn train/eval; đồ thị hash không vòng và không tự tham chiếu.

Field	Kiểu / null
contract_version	const alpr-contract-v1
manifest_version	const alpr-manifest-v1
status	enum TEMPLATE_NOT_A_RELEASE,READY
created_at	string | null
release_id	string | null
git_commit	string | null
protocol_version	const alpr-custom-base-group-v1
frozen_run_manifest_sha256	string | null
source_manifest_sha256	string | null
data_manifest_sha256	string | null
split_manifest_sha256	string | null
config_sha256	string | null
charset_sha256	string | null
detector_checkpoint_sha256	string | null
recognizer_checkpoint_sha256	string | null
runtime_inventory	array[Artifact]
evaluation_inventory	array[Artifact]
gate_decisions	object

Invariant liên record và điều kiện từ chối

• sample_id=image_id là opaque ID duy nhất; basename runtime không chứa annotation.

• Bbox: 0<=x1<x2<=W, 0<=y1<y2<=H, float pixel half-open; corners theo RB/LB/LT/RT.

• Mọi sample ID, pixel hash, dedup group và chuỗi plate NFC không giao nhau giữa các split.

• Chia split trước; crop/augmentation kế thừa ảnh cha, FK crop.sample_id phải resolve được.

• RuntimeImage/RuntimeCrop chỉ nhận pixel và ID. Trainer dùng GT train hợp lệ; test gold chỉ evaluator.

• NFC duy nhất. Sentinel O bị quarantine; blank0 không là target; charset/hash khớp output head.

• CTC T40, target7 baseline hoặc8 mở rộng; T>=L+repeat; input_lengths40 kể cả padding.

• Không dùng GT corners trong inference; oracle phải tách khỏi headline E2E.

• Failure pipeline có text rỗng; no_detection/low_confidence có selected_detection=null và ocr=null.

• READY từ chối schema/version/hash/architecture/shape/decoder không tương thích.

• SHA256 theo byte; hash tập ID theo JSON sort UTF-8 compact LF; manifests không vòng.

• TEMPLATE cho phép null nhưng loader phải từ chối; READY inventory không rỗng.

Đơn vị, hash và lifecycle

Sample.image_id=sample_id UUIDv4 sinh trong chuẩn bị rồi pin manifest; ID không chứa plate/text/bbox. image_sha256 là byte file nguồn runtime; pixel_sha256 là SHA256 của uint32 big-endian H,W nối RGB uint8 row-major, không gồm tên/metadata. Cùng pixel khác encoding vẫn có cùng pixel hash; same source có thể khác byte hash. Paths tương đối dưới release root; cấm absolute path, .., symlink thoát root; resolver không tự quét raw/evaluation. Metadata runtime chỉ ID, width/height lấy từ pixels; không source filename, nhãn, corners hoặc split gold.

Dataset hash giữ trong data manifest (inventory artifacts + counts + contract version; không self-hash). Split hash là hash file manifest cuối cùng; membership hash là JSON object train/dev/test với IDs sort, UTF-8 compact separators, LF cuối. Config hash theo exact YAML bytes đã freeze; charset hash là array tokens JSON UTF-8 compact +LF, checkpoint hash là exact file bytes. Run manifest trỏ các hash này, release manifest hash các manifest con. Không có vòng self-reference.

READY loader phải từ chối template, thiếu file, sai size/hash, sai contract/version, charset/head không khớp, normalization/blank/shape/time/decoder khác, split/config khác frozen release hoặc inventory chứa role cấm. Runtime inventory chỉ checkpoint/config/charset/runtime images và metadata pixel cần thiết; không Annotation, raw source locator, filename CCPD, train labels, dev/test gold hoặc expected output. Academic package có thể chứa evaluation inventory riêng nhưng predict() không đọc thư mục đó.

Low confidence: detector có bbox nhưng tất cả dưới conf runtime => status low_confidence, text empty; không có candidate => no_detection. Bbox không hợp lệ/crop rỗng => crop_error, không fallback GT crop. OCR decode rỗng => ocr_empty; lỗi tensor/decoder => ocr_error; invalid pixels => invalid_input; lỗi bất ngờ => runtime_error. Các lỗi có error_code, text empty và vẫn tính vào E2E/CER. ok yêu cầu selected_detection, OCR status ok và text nonempty; score OCR thấp chỉ hiển thị diagnostic, baseline không abstain thêm bằng OCR confidence. Không nối chuỗi từ nhiều bbox. Nếu GT text không supported trong data preparation thì quarantine trước freeze, không xóa ký tự để làm label hợp lệ.

Xem CTC/API, parser/split và test specification. Validator kế hoạch chỉ kiểm schema/template/cross-document; acceptance suite data/model thực tế do A/B/C/D triển khai sau, gồm mọi invariant liên record trên.

RunManifest.stage phân biệt data/detector/recognizer/pipeline: checkpoint chưa có ở giai đoạn data được null; detector READY đòi detector hash; recognizer READY đòi recognizer hash; pipeline READY đòi cả hai. created_at là ISO8601 UTC, size_bytes đơn vị byte, latency_ms đơn vị mili giây.

CCPD, parser và protocol chống rò rỉ

Phiên bản và phạm vi

Nguồn: CCPD chính thức (https://github.com/detectRecog/CCPD), loader (https://github.com/detectRecog/CCPD/blob/master/rpnet/load_data.py), split (https://github.com/detectRecog/CCPD/tree/master/split). MVP dùng archive **CCPD2019**, subset ccpd_base, 7 ký tự. Nguồn có bổ sung Green năm 2020, biển 8 ký tự; Green chỉ mở rộng version dữ liệu/allowed_lengths và báo metric riêng, không trộn ngầm vào Base. Chưa tải archive; A phải pin upstream commit/archive URL/hash và xác minh inventory trước G1.

Protocol nhóm **alpr-custom-base-group-v1**, seed42, không dùng official split cho headline MVP. Official README dùng Base cho train/val và các subset khó cho test; protocol detection gốc chọn 1 bbox và IoU>0.7. Nhóm dùng Base group split và P/R/AP@0.50 cùng mAP50-95; vì khác dữ liệu lẫn metric nên không tuyên bố so ngang benchmark gốc. Nếu chạy official protocol bổ sung, dùng manifest riêng, metric joint@0.7 với bất đẳng thức đúng nguồn, không đổi tên kết quả custom thành official.

Mức	Train	Dev	Test	Dùng để
Mục tiêu	20.000	2.000	2.000	Chất lượng baseline, nếu đủ dung lượng/GPU
Tối thiểu	5.000	500	500	MVP nhỏ, vẫn group-disjoint
Pilot từ split đã khóa	1.000	200	Không có pilot test	Tốc độ/VRAM và dev sanity

Đây là số mẫu mục tiêu, chưa phải counts đã quan sát. Test được chọn trước huấn luyện, giữ nguyên trong suốt pilot/scaling. Nếu giảm từ mục tiêu xuống tối thiểu: chỉ lấy train/dev subset theo thứ tự seed cố định; test đã freeze giữ nguyên nếu đã đủ tải. Nếu buộc đổi test inventory do nguồn lỗi trước exposure thì tạo protocol/split version mới và decision.

Parser filename chỉ dành cho data/evaluator

Tên nguồn tách đuôi extension rồi split - thành đúng 7 field: area ratio, tilt horizontal/vertical, bbox, 4 corners, plate indices, brightness, blur. Area/tilt/brightness/blur giữ raw field cho stratification; không dùng chúng làm feature inference. Không giả định đơn vị số metadata nếu chưa xác minh trong archive/paper; bin khó định nghĩa bằng quantile train và pin ngưỡng trong manifest, không điều chỉnh theo kết quả test.

1. Bbox field x1&y1_x2&y2, corners bốn x&y theo **RB → LB → LT → RT**; nguồn tọa độ ảnh gốc, x ngang/y dọc, origin góc trái trên. Preserve raw để audit. 2. Canonical bbox float [x1,y1,x2,y2) half-open. Vì nguồn dùng hiệu right-left, baseline không cộng 1; A kiểm overlay 100 ảnh trước freeze, nếu điều chỉnh convention phải version/parser và regenerate mọi crop/label. 3. Clip rounding chỉ khi tạo crop: left/top=floor, right/bottom=ceil, clamp [0,W]/[0,H]. Annotation ngoài bounds hoặc x1>=x2/y1>=y2 bị quarantine, không lặng lẽ sửa nhãn; prediction out-of-bounds được clamp trong runtime với log; nếu còn rỗng => crop_error. Corners invalid/self-intersect bị quarantine. 4. Plate indices: vị trí0 dùng provinces, vị trí1 dùng alphabets, còn lại dùng ads. Mapping đầy đủ nằm ở x-annotation-mapping trong JSON contract. Kiểm index không âm/không vượt giới hạn, dài7 MVP. Đối chiếu sample chính thức: 0_0_22_27_27_33_16 → 皖AY339S (fixture nguồn, không phải prediction). 5. Ký tự cuối bảng nguồn là chữ **O sentinel "không ký tự"**, không phải số0. Sentinel không phải CTC blank, không target hợp lệ; quarantine với reason=sentinel_annotation. Không xóa sentinel rồi gán nhãn ngắn hơn. Index0 province là 皖, không phải blank. 6. UTF-8, normalize NFC duy nhất; không đổi case/strip ký tự để sửa nhãn. Charset ccpd67-v1 có 33 token tỉnh/đặc biệt +24 chữ Latin (không I/O) +10 số; CTC thêm blank0 => head68. Charset mẫu là thiết kế vocabulary, không lấy từ test. 7. Ảnh decode lỗi, dimensions sai, filename sai field, bbox/corners/index lỗi, length8 trong MVP, unsupported token => quarantine JSONL gồm source locator/hash/reason/parser_version; báo counts từng reason. Lấy mẫu audit quarantine bằng train/dev hoặc source QA trước freeze; không chọn test dễ hơn qua kết quả model.

Split, dedup và giới hạn cùng xe

A lập inventory ảnh nguồn một lần, sample_id UUID opaque và source mapping chỉ trong evaluator/preparation. Chạy trước sampling/split: SHA256 file, hash decoded RGB; pHash64 toàn ảnh và plate crop GT cho candidate near-duplicate, Hamming<=4 dùng tìm candidate, không tự coi mọi candidate là trùng. Review candidate pairs và ghi bằng chứng; đồng thời union mọi ảnh có plate_text NFC trùng. Toàn connected component thành dedup_group_id. Exact duplicate giữ representative theo source locator sort, log alias/lý do loại; near-duplicate giữ cùng nhóm, không cần xóa hết. Ảnh augmentation chưa được sinh ở bước này.

Để tránh O(N²) toàn dataset, A dùng hash bucket/BK-tree hoặc indexing theo pHash và review candidates trên pool Base đã chọn rộng hơn quota. Sampling chỉ từ component và pin pool hash; nói rõ kiểm near-duplicate giới hạn trong pool, không tuyên bố chống rò toàn CCPD. Candidate chưa phân xử thì gộp bảo thủ hoặc quarantine, không tách qua split. Biển giống nhau nối nhóm kể cả góc/độ sáng khác. Điều này giảm text memorization nhưng không chứng minh cùng xe: không có vehicle ID đáng tin để phát hiện xe đổi biển/nhãn sai hoặc mọi cảnh gần trùng.

Thứ tự deterministic: sort group ID, RNG seed42 shuffle; phân nhóm vào test, dev, train theo quota ảnh xấp xỉ, giữ nguyên component, cho phép vượt quota do kích thước nhóm và ghi actual N. Chọn thêm nhóm từ cùng pool nếu thiếu minimum, không bẻ component; log phân bố province/blur/tilt/brightness. Leakage report bắt buộc pairwise train-dev/train-test/dev-test: overlap sample, byte hash, pixel hash, dedup group và text phải 0; unresolved near-duplicate cross-split phải 0. Chỉ sau đó crop, augment train và export detector labels. Crop/augment kế thừa parent split. Không shuffle tất cả crop rồi split lại.

Ranh giới nhãn và test exposure

Trainer được nhận GT train, dev evaluator nhận dev GT để chọn model, early stop và tham số. A cần parse test nguồn để chuẩn bị split và audit tự động overlap; không dùng test pixels/labels để thiết kế augmentation, chọn model/decoder hoặc manual QA chọn ảnh đẹp. Tạo test bundle riêng, exposure log ghi A đã thấy metadata lúc chuẩn bị. B/C không được dùng test labels; D chỉ mở sau frozen checkpoint/config/decoder/split hashes.

Preparation copy/re-encode lossless nếu cần để có basename UUID, ghi hash pixel trước/sau và metadata encoding; runtime loader nhận pixels RGB + ID, không source filename hoặc raw path chứa annotation. Test renaming: 100 dev images, giữ pixel y hệt, đổi basename và opaque ID, chạy predict 2 lần; bbox/score/text/status không đổi (bỏ trường ID); deterministic CUDA nếu có hoặc tolerance score<=1e-6 đã freeze, text và status phải exact. Thêm test các filename cố ý chứa GT sai để bắt code parse lén. Runtime dependency graph không import parser, không đọc evaluator gold hoặc raw cache. Không sử dụng GT corners ở pipeline. Warp GT corners là oracle thí nghiệm riêng nếu còn thời gian.

Kiến trúc, CTC và giao diện module

Detector và crop

Baseline **YOLOv8n detection**, package **ultralytics==8.3.0**, một lớp plate, image640, batch16 tham khảo, seed42. Đây là lựa chọn baseline nhỏ để hạn chế chi phí thử nhiều model; không khẳng định là phiên bản mới nhất. Chưa kiểm tương thích trên máy/GPU này. Nguồn YOLOv8 (https://docs.ultralytics.com/models/yolov8/), release v8.3.0 (https://github.com/ultralytics/ultralytics/releases/tag/v8.3.0). Torch2.5.1/torchvision0.20.1 với Python3.11 là đề xuất; B phải smoke và khóa lock/CUDA trước train.

Input uint8 RGB HxWx3; thư viện nhận BGR thì adapter đổi đúng một lần. Detector làm letterbox640, trả bbox original pixels float, clip, class0, score. detect() trả candidates sau NMS với score>=0.001; orchestrator áp runtime conf0.25 để phân biệt low_confidence/no_detection. Runtime NMS IoU0.50, conf0.25 baseline; AP confidence0.001 là evaluator diagnostic riêng. NMS stable sort theo score và bbox để deterministic. Top1 chọn score cao nhất sau NMS/threshold, tie bbox lexicographic; không dùng GT/recognizer để chọn. Crop rectangle dự đoán, padding bbox=0 baseline; box margin5% là extension khác chỉ chọn trên dev. Rectification không nằm baseline; nếu có cần predicted corners model riêng, còn GT corners chỉ oracle.

Recognizer bắt buộc CNN + BiLSTM + CTC

CRNN gốc (https://arxiv.org/abs/1507.05717) cung cấp hướng kiến trúc; dưới đây là thiết kế nhỏ của nhóm, không tuyên bố tái hiện chính xác kiến trúc/thành tích bài báo.

Bước	Tensor / cấu hình
Ảnh crop RGB	uint8 HxWx3, không text/bbox GT trong runtime input
Grayscale	round(0.299R+0.587G+0.114B), clamp uint8
Resize/pad	H=32; wr=min(160,max(4,round(Wcrop*32/Hcrop))); resize bilinear 32xwr, pad phải bằng pixel127 lên160
Normalize	float32 /127.5 -1; shape N x1 x32 x160
CNN block1	Conv3x3 pad1 1→64 +ReLU; MaxPool2x2 stride2 → N x64 x16 x80
CNN block2	Conv3x3 pad1 64→128 +ReLU; MaxPool2x2 stride2 → N x128 x8 x40
CNN block3	Conv3x3 pad1 128→256 +ReLU; MaxPool(2,1) → N x256 x4 x40
CNN block4	Conv3x3 pad1 256→256 +ReLU; MaxPool(2,1) → N x256 x2 x40
Collapse height	Conv(2,1), no padding, 256→256 +ReLU → N x256 x1 x40
Sequence	squeeze height, permute → N xT40 x256
BiLSTM	2 layers, hidden128 mỗi chiều, batch_first=True, output N x40 x256
Head	Linear256→C68, log_softmax cuối → N x40 x68; transpose T40 xN x68 cho loss

Ảnh quá rộng bị resize về 32x160 có distortion; report wr clipped rate, không warp bằng GT corners. Toàn bộ T=40 timestep được tính, kể cả padding phải; input_lengths=[40]*N, cùng quy tắc lúc train và decode. Không dùng ceil(wr/4) với CNN này mà chưa đổi contract, vì receptive field/padding cần kiểm riêng. Targets là concatenated 1D int64 không blank, target_lengths=[7]*N MVP, mở rộng length8 version riêng. L là độ dài target, R là số cặp ký tự bằng nhau kề nhau; cần **T>=L+R** để CTC có alignment. T40 đủ với 7/8 ký tự kể cả repeat; vẫn test shape và bound, reject invalid sample trước loss. CTCLoss(blank=0,reduction='mean',zero_infinity=False) baseline, tính log_softmax fp32, target/input lengths đúng batch; padding target nếu dùng phải có length mask rõ. Không bật zero_infinity để giấu lỗi lengths. Loss finite và backward nonzero trên batch synthetic/train là kiểm bắt buộc, không là benchmark recognition.

Greedy CTC: argmax từng t → collapse consecutive token lặp → xóa blank0. Ví dụ [A,A,blank,A,B,B] → AAB; nếu xóa blank trước collapse sẽ sai repeated character. Sentinel O của annotation bị loại khỏi training theo quarantine; CTC blank là token kỹ thuật riêng index0. Decode không dùng luật biển/beam/GT dictionary để sửa output baseline. Confidence OCR = trung bình max softmax trên timestep nonblank của greedy path, null nếu không có nonblank; score không calibrated, chỉ diagnostic, không ngầm loại mẫu khỏi đánh giá.

Giao diện cụ thể (chưa có implementation)

interfaces.py chứa Protocol/dataclass, không sinh prediction.

```
detect(image: RuntimeImage) -> Sequence[DetectionPrediction]
recognize(crops: Sequence[RuntimeCrop]) -> Sequence[OCRPrediction]
predict(image: RuntimeImage) -> PipelineResult
evaluate(predictions, ground_truth, *, split_manifest, mode) -> MetricReport
```

RuntimeImage(sample_id, pixels) và RuntimeCrop(crop_id,sample_id,pixels) chỉ chứa opaque IDs/pixels. Detector return [] khi không có candidates; orchestrator phân biệt no_detection và low_confidence. recognize giữ thứ tự input, trả một record/crop gồm empty/error status. predict trả đúng một result/ảnh, gắn selected detection/OCR hoặc null, text empty cho failure. Batch wrapper bắt lỗi từng ảnh và giữ toàn bộ ID. MetricReport dự kiến: mode, split_hash, Nimages, Ngt_chars, coverage, failure_counts, metric objects gồm numerator/denominator/value/unit, model/config/charset hashes, per_image_artifact. N=0 => value null + diagnostic; duplicate/unknown IDs là lỗi evaluator, không quietly inner-join.

Mode evaluator: detection; ocr_gt_bbox; ocr_pred_matched; pipeline_all. GT chỉ đi vào evaluator/trainer, không nhận qua predict. Evaluator nối predictions/GT bằng sample_id, không row order và không parse path runtime. Training dataset có thể dùng annotations được chuẩn bị hợp lệ; nguyên tắc pixel-only áp dụng inference.

Checkpoint và khả năng nạp lại

Detector .pt cùng model manifest/Ultralytics version; B xác minh cách load phù hợp version đã pin. CRNN .pt: dictionary gồm format_version, architecture_id=crnn-small-v1, state_dict, input_shape=[1,32,160], time_steps=40, num_classes=68, blank_index=0, charset_sha256, preprocess_version, decoder_version=greedy-ctc-v1, training config hash, epoch và seed. Optimizer/scheduler resume artifact riêng; final inference chỉ cần state_dict và metadata. Checkpoint external metadata phải khớp manifest; không unpickle checkpoint không biết nguồn. Load mismatch head/charset/blank/shape/preprocess/version/hash => từ chối rõ ràng trước inference, không tự tải checkpoint thay thế hoặc âm thầm khởi tạo mạng random.

Tài nguyên, ngân sách và sổ rủi ro

Quan sát môi trường ngày 08/11/2026

Python3.14.3; os.cpu_count()=8 logical CPU; ổ F còn khoảng **25,55 GiB** tại thời điểm khảo sát. Windows CIM CPU/RAM/GPU bị Access denied; không tìm thấy nvidia-smi trong PATH. Chưa xác minh model CPU, RAM, GPU, VRAM hoặc CUDA. Không suy ra máy không có GPU từ việc thiếu nvidia-smi. PyYAML6.0.3, jsonschema4.26.0, ReportLab5.0.1 và font Arial có sẵn; không có pandoc/xelatex trong PATH. Python3.14 chỉ dùng kiểm tài liệu; tạo môi trường training Python3.11 sau, chưa cài torch.

Ba loại thời gian tách biệt

Loại	Ngân sách/ước lượng ban đầu	Cách xác minh
Giờ công	4 người x40h =160h; 136 thực hiện +24 reserve	Backlog tổng 34+6 mỗi người; chờ GPU không tính thành giờ thao tác
Tải dữ liệu	Chưa biết, phụ thuộc archive thật/băng thông; ví dụ 10GB ở10MB/s lý tưởng ~17 phút, không là ETA cam kết	A đo size/bandwidth/retry; cấp slot tải riêng ngoài giờ công active
GPU detector	Pilot <=0,5h tham khảo; full subset trần12 GPU giờ, nằm trong tuần B	Đo seconds/epoch trên pilot và ngoại suy theo ảnh/epochs, margin x1.5
GPU CRNN	Pilot <=0,5h tham khảo; full subset +1 ablation trần16 GPU giờ, nằm trong tuần C	Đo throughput thực tế/VRAM rồi chốt số epoch; không hứa hội tụ trước pilot

Giả định tham khảo: một GPU CUDA VRAM>=8GB, RAM>=16GB, disk ngoại vi/remote nếu full archive không vừa. Các con số GPU là giới hạn chi phí đề xuất, không là benchmark đã chạy hoặc dự báo chính xác. Download và training chạy nền phải hoàn thành trong cửa sổ tuần tương ứng; nếu dự báo quá tuần thì giảm scope hoặc đổi lịch. Mỗi người giữ 6h reserve cho lỗi nguồn/tensor/integration; không phân reserve cho video hay production API.

Pilot và giảm phạm vi

1. A kiểm archive size + extracted subset + crop + checkpoints + workspace overhead trước tải. Giữ >=8GiB headroom; với disk hiện tại, chặn tải nếu dự kiến working set vượt ~17GiB. Chưa biết kích thước archive thì không mặc định đủ. Không tải CCPD toàn bộ trong bước lập kế hoạch. 2. Thu subset Base qua nguồn chính thức nếu có lựa chọn; nếu archive bắt buộc tải cả, dùng volume ngoài/compute workspace được cấp và chuyển subset có provenance. Không tự giả định hỗ trợ HTTP range extraction. 3. Freeze split group trước, pilot 1k/200 từ train/dev. Đo parse time, batch memory và epoch time; image decode/crop on-demand thay cache toàn crop khi disk chật. Raw không bao giờ bị xóa trước xác minh bản nguồn an toàn. 4. Nếu target20k không vừa: dùng minimum5k/500/500, batch giảm theo VRAM, early-stop và số epoch giới hạn. B/C chỉ tối đa2 cấu hình; không sweep. Thời gian ablation nằm trong cùng budget. 5. CPU-only: parser/evaluator/smoke chạy được; xin slot GPU/đổi lịch là blocker training thật. Không cam kết đạt quality trên CPU hoặc thay CRNN bằng OCR bên ngoài. Giữ status CHƯA ĐẠT nếu thiếu compute. 6. Cắt extension theo thứ tự: video/API/tracking → beam/rectification → subset khó/Green. Baseline YOLO+CRNN+CTC, full-split metric và anti-leak không được cắt. Ablation bắt buộc thiếu phải ghi fail, không giả hoàn thành.

Risk register

ID	Rủi ro / tín hiệu	Chủ sở hữu	Xử lý / điều kiện chặn
R01	Disk <working set+8GiB	A	Subset/on-demand hoặc volume ngoài; dừng tải lớn chưa đủ chỗ
R02	Không có GPU/wheel Python	B	Python3.11 env riêng, smoke torch/CUDA; chỉ CPU smoke và báo blocker training
R03	Sai mapping/sentinel/corners	A	QA100+fixtures, quarantine; G1 fail khi còn mơ hồ
R04	Text/near-duplicate overlap	A	Union groups trước split; overlap>0 chặn G1
R05	Filename leakage	C	Pixel-only API, rename test và module isolation; bất kỳ lệ thuộc nhãn chặn G3
R06	CNN timestep/CTC collapse sai	C	Shape/repeat/length tests, overfit32 train; không dùng zero_infinity che lỗi
R07	Predicted crop giảm OCR	B/C	Báo GT vs pred/coverage; dev-only preprocessing; giữ E2E đủ mẫu số
R08	Tuần C quá tải	B/C	A viết metrics/fixtures, B giao adapter/pred crops; ghép predict từ đầu C; cắt extensions
R09	Test bị dùng tuning	D	Exposure log + freeze hashes; nếu lộ ghi hạn chế, không gọi held-out chưa xem
R10	Người giao hết tuần, artifact lỗi	Người nhận	Từ chối gate, reserve/đổi lịch rõ; không giả định hỗ trợ ngoài quỹ
R11	Model/license không phân phối được	B/D	Ghi nguồn/license trước tải/đóng gói; cung cấp script lấy artifact thay commit dữ liệu lớn
R12	Ngưỡng pilot thiếu cơ sở	B/C	Hiệu chỉnh từ dev có decision trước test; không đưa paper score thành cam kết

Risk register hiện là rủi ro dự kiến; chưa có log training hoặc measured throughput.

Tái lập và phân biệt trạng thái

Lệnh hiện có

Chạy từ root repository:

```
python -m pip install -r requirements-docs.txt
python scripts/validate_plan.py
python docs/plan/build_plan.py --check
python docs/plan/build_plan.py
```

Install command là hướng dẫn khi máy thiếu dependency; môi trường lập kế hoạch đã có sẵn nên không cài lại. Validator kiểm link Markdown local, JSON strict, JSON Schema và manifest/charset templates, YAML, CSV, benchmark table/JSON/template, versions/ownership/budget và null chưa đo. Validator **không kiểm ảnh thật, chống rò dataset thật hoặc chất lượng mô hình**. Build --check kiểm dependency/font/docs; build tạo reports/plan/ALPR-Implementation-Plan.pdf thật, gồm README plan và 00–11, decision log, nguồn; in số trang/hash/kích thước sau khi tạo. Arial cho tiếng Việt, STSong-Light cho ký tự CJK; nếu máy thiếu font/dependency thì script fail rõ và giữ Markdown đầy đủ, không ghi PDF rỗng. Có thể dùng --output để chọn đường dẫn.

Giao diện lệnh tương lai — CHƯA TRIỂN KHAI

Không copy chạy các lệnh dưới ở skeleton này. Tên CLI là đề xuất, chủ sở hữu có thể đổi nhưng phải cập nhật command registry/biên bản và giữ input/output tương đương.

```
python -m alpr.cli prepare-data --config configs/frozen.yaml
python -m alpr.cli train-detector --config configs/frozen.yaml
python -m alpr.cli evaluate --mode detection --split dev --config configs/frozen.yaml
python -m alpr.cli export-crops --kind predicted --split dev --config configs/frozen.yaml
python -m alpr.cli train-recognizer --config configs/frozen.yaml
python -m alpr.cli evaluate --mode ocr_gt_bbox --split dev --config configs/frozen.yaml
python -m alpr.cli evaluate --mode ocr_pred_matched --split dev --config configs/frozen.yaml
python -m alpr.cli evaluate --mode pipeline_all --split test --config configs/frozen.yaml
python -m alpr.cli predict --image data/processed/images/opaque-id.jpg --config configs/frozen.yaml
python -m alpr.cli latency --split dev --config configs/frozen.yaml
python -m alpr.cli verify-gate --gate G3 --config configs/frozen.yaml
```

Future train commands chỉ dùng train/dev; test evaluate chỉ sau C freeze và D kiểm nhận. Future config configs/frozen.yaml và runtime data/model path chưa tồn tại vì chưa có run thật. Copy config example sau pilot, điền hash đã đo; không dùng null template làm release thực thi. Mỗi verify-gate phải có dispatch cho mọi benchmark ID trong JSON hoặc cung cấp lệnh thay thế đã chạy thật. PYTHONPATH=src hoặc editable install dùng cho triển khai sau; hiện chưa có alpr.cli.

Bảng trạng thái ban đầu

Thành phần	Hiện trạng	Evidence
Kế hoạch/schema/config/templates	ĐÃ TẠO	Các file trong docs/plan
Protocol Python	ĐÃ TẠO, chỉ interface	src/alpr/interfaces.py
Validator/PDF builder	ĐÃ TRIỂN KHAI	scripts/validate_plan.py; docs/plan/build_plan.py
Parser/split/crop/charset dataset release	CHƯA TRIỂN KHAI	Không có artifact thật
YOLO detector/CRNN/training	CHƯA TRIỂN KHAI	Checkpoint null
Pipeline/evaluator/demo	CHƯA TRIỂN KHAI	Không có prediction hay report mô hình
CER/E2E/P-R/mAP/latency	CHƯA ĐO	Template null

Lịch sử lệnh thực sự đã chạy và kết quả kiểm tài liệu nằm ở biên bản kiểm tra. PDF là bản kế hoạch, không chứng minh huấn luyện hay gate model đã đạt.

Đặc tả kiểm thử ALPR cần triển khai

Hiện **chưa có test/model ALPR thực thi**. scripts/validate_plan.py kiểm tài liệu, không chứng nhận các ca dưới đã pass. Fixtures synthetic hoặc train/dev; không lấy test images làm regression tuning. Khi triển khai, pytest hoặc unittest đều được.

Nhóm	Chủ sở hữu	Ca có ý nghĩa / expected behavior
Parser	A	Filename nguồn mẫu → 皖AY339S; bbox154,383,386,473; corners RB,LB,LT,RT đúng thứ tự
Parser invalid	A	Sai7 fields, âm/out-of-range indices, sentinel O, unsupported length8, bbox ngược, corrupt image → quarantine có reason
Split	A	Duplicate byte/decoded pixel, same plate, near-pair thành chain union → chỉ một split; crop giữ parent split
Pixel hash	A	Cùng RGB/dim khác tên/encoding → cùng pixel hash; khác dimensions hoặc pixel → khác hash
Charset	A/C	Index0 province không thành CTC blank; head68; tokens/hash/NFC đúng; sentinel không target
Metrics	A	GT ABC, pred ADC => edit1, CER1/3; GT AA pred empty => edit2; micro (1+2)/(3+2)=0.6
Detection	A/B	Identity IoU1, disjoint0, boundary half-open, duplicate detection chỉ match GT một lần
CTC	A/C	[A,A,blank,A,B,B]→AAB; all blank→empty; T<L+repeat reject; shape T40 và target không blank
CRNN loss	C	Batch có repeated target, finite CTCLoss/backward, gradient nonzero; mismatch charset checkpoint reject
Pipeline	B/C	Top1 tie policy stable; undo letterbox; conf miss; crop clamp/error; OCR empty/error không crash batch
Evaluator	C/D	3 ảnh gồm 1 correct, 1 miss, 1 wrong → E2E1/3; miss đóng đủ GT deletions trong CER
Evaluator coverage	C/D	Missing prediction tính empty/fail và báo lỗi; duplicate/unknown IDs fail; Nmatched0 => conditional null
Anti-leak	C	Đổi basename và ID giữ pixel, label giả trong tên → text/bbox/status không đổi; inference không import parser/GT
Artifacts	B/C/D	Thiếu/mismatch checkpoint/config/data/split/charset hash, TEMPLATE status, path traversal → reject load
Demo	D	6 fixture G4.1, offline reload; không tự tải model/network hoặc giấu ảnh lỗi

Ngưỡng tối thiểu fixture tổng G1.5 >=20 và G3.1 >=8 không thay coverage các nhóm bắt buộc. Đếm fixture thực tế và expected assertion; không tạo nhiều bản cùng một test chỉ để đạt số lượng.

Nhật ký quyết định

Các quyết định dưới là **thiết kế ban đầu**, ngày 08/11/2026; không là kết quả pilot. Chưa có ngày khởi động cụ thể. A/B/C/D là vai trò dự kiến, chưa có chữ ký nghiệm thu.

ID	Quyết định	Lý do	Trạng thái / thời điểm xem lại
DEC01	CCPD2019 Base, custom group split seed42; target20k/2k/2k, min5k/500/500	Vừa ngân sách, giảm text memorization	Đề xuất; A pin inventory/hash trước G1
DEC02	YOLOv8n / ultralytics8.3.0, image640, train riêng	Baseline nhỏ, giảm số nhánh thử	Đề xuất; B smoke dependency/GPU trước train
DEC03	CRNN-small CNN stride4 +BiLSTM2x128 +CTC, input32x160/T40/head68	Đáp ứng đề, dễ kiểm alignment	Đề xuất; C kiểm shape và finite backward
DEC04	Crop chữ nhật, greedy CTC, không GT warp trong runtime	Deployment chỉ có pixel	Baseline; sửa phải version/config và dev evidence
DEC05	A làm metric/fixture, B giao adapter/crop, C giao predict() thật	Giảm overload C và tránh D ghép core	Ràng buộc bàn giao
DEC06	Quality thresholds provisional, integrity bắt buộc	Chưa có GPU hoặc internal baseline	B/C pilot dev rồi freeze; chưa từng xem test model
DEC07	Preprocessing ablation duy nhất CLAHE so baseline	Vừa budget hai cấu hình	Chọn trên dev; D chỉ đo final frozen variants
DEC08	Giữ test cố định khi giảm train scope	Tránh đổi test theo kết quả	Chỉ sửa test lỗi nguồn trước exposure, có protocol version mới

Template quyết định tiếp theo

• ID, ngày, tác giả/chủ sở hữu/người nhận:

• Đổi gì: data/split/config/architecture/threshold/protocol version cũ → mới:

• Bằng chứng dev/pilot, hashes, denominator, resource logs:

• Tác động công sức/GPU/disk và API/bàn giao:

• Test đã được ai xem, xem gì, khi nào; xác nhận trước test exposure nếu là tuning:

• Kết luận và freeze timestamp/hash; ngưỡng nào trở thành cam kết:

• Nếu sửa correctness sau test: error exposure, mọi run cần chạy lại, giới hạn kết luận:

Không được hạ ngưỡng sau khi biết test thấp, đổi tên split để giấu exposure hoặc điền observed=0 khi chưa chạy.

Nguồn và cách vận dụng repo tham khảo

Đọc ngày 08/11/2026, bằng git show và git ls-tree, không checkout/sửa repo mẫu. Repo local: F:/HUST/20261/WebMining/Solving-the-Semantic-Attribute-Gap-in-E-Commerce-Product-Search.

Ref	Commit đã đọc	Nội dung áp dụng
main	ffc228b0f3a6bde7ccc3bb6994fb45ac3aa6059d	README rất ngắn, dùng root làm điểm vào tài liệu
origin/plan	1fd661e68e84f5a2cef0310e565b44304740fc10	docs/plan/00–06, tuần 1–4, JSON, CSV, config, template, build_plan.py
origin/feature/data_processing	e7b41a9e640711d1bd662b4fe60f750517dd8755	src/, data/, evaluation/, manifests/, handoff/, reports/, README, smoke guide

Không tìm thấy AGENTS.md trong cây ba ref hoặc các thư mục cha workspace đã kiểm. Đọc thêm handoff W1_A_to_B, runtime manifest và config của nhánh data_processing. Nhánh triển khai có artifact/số đo riêng; chúng không chứng minh tiến độ ALPR. Giữ gate có số đo/bằng chứng, manifest bất biến, runtime/evaluator tách biệt, smoke offline và trách nhiệm người nhận. Đổi toàn bộ schema sang ảnh/bbox/crop/chuỗi/CTC; không dùng lại ontology/retrieval metric của dự án cũ.

Nguồn kỹ thuật chính thức

• CCPD README (https://github.com/detectRecog/CCPD): phiên bản 2019, Green bổ sung 2020, filename annotation,

mapping và protocol một biển mỗi ảnh. Ngày đọc 08/11/2026; chưa pin commit upstream hoặc tải archive.

• CCPD split (https://github.com/detectRecog/CCPD/tree/master/split): đối chiếu official protocol;

MVP dùng custom split đã đặt tên riêng, không so trực tiếp bảng kết quả chính thức.

• CCPD loader nguồn (https://github.com/detectRecog/CCPD/blob/master/rpnet/load_data.py): đối chiếu bbox/index,

chỉ tham khảo chuẩn bị dữ liệu; không đưa parser filename vào inference.

• Bài báo CCPD (https://openaccess.thecvf.com/content_ECCV_2018/html/Zhenbo_Xu_Towards_End-to-End_License_ECCV_2018_paper.html):

Xu et al., ECCV 2018, nguồn gốc dataset và baseline; không lấy kết quả bài báo làm cam kết nhóm.

• YOLOv8 (https://docs.ultralytics.com/models/yolov8/) và

Ultralytics v8.3.0 (https://github.com/ultralytics/ultralytics/releases/tag/v8.3.0): chọn YOLOv8n thay vì dò nhiều kiến trúc, nhỏ và có adapter detection sẵn; phiên bản baseline được đề xuất cố định.

• CRNN (https://arxiv.org/abs/1507.05717): Shi, Bai, Yao, CNN + recurrent sequence modeling + transcription.

• PyTorch CTCLoss source v2.5.1 (https://github.com/pytorch/pytorch/blob/v2.5.1/torch/nn/modules/loss.py):

API log-probabilities (T,N,C), input/target lengths và blank. Công thức shape/time cụ thể là thiết kế của nhóm.

A phải ghi commit CCPD, URL tải thật, ngày tải, SHA-256, phiên bản archive và license trong source manifest. Chưa pin upstream không được trình bày là đã có provenance của dữ liệu. Phiên bản thư viện training là đề xuất để pilot, không phải dependency lock đã kiểm.